SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
 <input type="text" name="name">
 <input type="email" name="email">
 <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.

5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
 <title>Company</title>
 </head>
 <body>
 <div>
 <h1>Welcome
 <p>About us
 </div>
 </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

*I, **Aaron Samuel S(192411022)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary*

action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure

The given HTML lacks semantic elements that improve structure and accessibility.

Missing semantic elements:

- `<header>` – Displays page title/logo.
- `<main>` – Contains the main content.
- `<section>` – Groups the registration form.
- `<form>` (already present)
- `<label>` – Associates labels with input fields.
- `<footer>` – Displays copyright/contact information.

2. Modify the structure to make it standards-compliant

```
 <!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Symposium Registration</title>

</head>

<body>

<header>

    <h1>Online Symposium Registration</h1>

</header>

<main>

    <section>

        <form action="/register" method="POST">
```

```
            <label for="name">Name:</label>
            <input type="text" id="name" name="name" required>
            <label for="email">Email:</label>
            <input type="email" id="email" name="email" required>
            <button type="submit">Submit</button>
        </form>
    </section>
</main>
<footer>
    <p>&copy; 2026 College Symposium</p>
</footer>
</body>
</html>
```

3. Interpret the HTTP request generated during form submission

When the user clicks Submit, the browser sends an HTTP POST request.

Example Request

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

name=John&email=john@example.com

Explanation

- POST → Sends data securely in the request body.
- /register → Server endpoint.
- Host → Server domain.
- Content-Type → Format of submitted data.
- Body → Contains user details.

4. Analyze the HTTP response cycle from server to browser

The communication follows these steps:

1. User fills the registration form.
2. Browser sends an HTTP POST request.
3. Server receives and validates the data.
4. Server stores the registration in the database.
5. Server sends an HTTP response.
6. Browser displays the response.

Example Response

HTTP/1.1 200 OK

Content-Type: text/html

Registration Successful

Possible status codes:

- 200 OK – Success.
- 201 Created – Resource created.
- 400 Bad Request – Invalid input.
- 500 Internal Server Error – Server issue.

5. Suggest improvements for usability and accessibility

- Use `<label>` for all input fields.
- Add the `required` attribute.
- Use placeholder text where appropriate.
- Display validation error messages.
- Use semantic HTML (`header`, `main`, `section`, `footer`).
- Ensure keyboard accessibility.
- Use sufficient color contrast.
- Make the page responsive using the viewport meta tag.

2. Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags

Issues

- **Missing `<!DOCTYPE html>`.**
- **Missing `lang` attribute.**
- **`<h1>` is not closed.**
- **`<p>` is not closed.**
- **Missing semantic elements.**
- **Improper nesting causes invalid HTML.**

---

2. Analyze how different browsers may interpret this structure

Different browsers use error recovery differently.

Possible outcomes:

- **Chrome may automatically close `<h1>` before `<p>`.**
- **Firefox may render the page slightly differently.**
- **Edge and Safari may also auto-correct the HTML.**
- **Layout inconsistencies occur because browsers fix invalid HTML differently.**

Hence, invalid HTML leads to inconsistent rendering.

---

3. Identify missing semantic elements

Missing elements include:

- **`<header>`**
- **`<main>`**
- **`<section>`**
- **`<footer>`**

These improve readability, accessibility, and SEO.

---

4. Provide a corrected W3C-compliant HTML structure

```
<!DOCTYPE html>

<html lang="en">


<head>

   <meta charset="UTF-8">

   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>Company</title>

</head>
```

```
<body>

<header>
    <h1>Welcome</h1>
</header>

<main>
    <section>
        <p>About us</p>
    </section>
</main>

<footer>
    <p>Copyright © 2026 Company</p>
</footer>

</body>
</html>
```

---

5. Suggest techniques to ensure cross-browser compatibility

- **Use valid HTML5 and CSS.**
- **Validate code using the W3C Validator.**
- **Include `<!DOCTYPE html>`.**
- **Test on Chrome, Firefox, Edge, and Safari.**
- **Use CSS reset or normalize stylesheets.**
- **Avoid browser-specific features unless necessary.**
- **Use responsive design.**
- **Use feature detection instead of browser detection.**

3. Form Submission Failure

1. Identify issues in the form configuration

Problems include:

- **Button type is `"button"` instead of `"submit"`.**
- **Password is being sent using the GET method.**
- **GET exposes sensitive information in the URL.**
- **No validation attributes like `required`.**

2. Analyze why the HTTP request is not triggered

The button is defined as:

<button type="button">Login</button>

A button with `type="button"` performs no default action.

Only

<button type="submit">

or JavaScript calling

form.submit();

will submit the form.

Therefore, clicking the button does not generate an HTTP request.

---

3. Interpret the role of GET method in this scenario

GET:

- **Appends data to the URL.**
- **Used for retrieving data.**
- **Data is visible in browser history and server logs.**
- **Not suitable for passwords or confidential information.**

Example:

/submit?username=John&password=12345

This is insecure.

---

4. Propose a corrected implementation

```
<!DOCTYPE html>

<html lang="en">


<head>

   <meta charset="UTF-8">

   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>Login</title>
```

```
</head>

<body>

<form action="/submit" method="POST">

    <label for="username">Username</label>

    <input type="text" id="username" name="username" required>

    <label for="password">Password</label>

    <input type="password" id="password" name="password" required>

    <button type="submit">Login</button>

</form>

</body>

</html>
```

---

5. Suggest improvements for secure data transmission

- **Use the POST method instead of GET.**
- **Enable HTTPS to encrypt communication.**

- **Validate inputs on both client and server.**
- **Hash passwords before storing them.**
- **Use strong authentication and session management.**
- **Protect against CSRF attacks using CSRF tokens.**
- **Sanitize user input to prevent SQL Injection and XSS.**
- **Display generic login error messages without revealing sensitive information.**